

FastAPI as a Microservice Gateway

Presentation Manuscript — Speaker Notes

~10 minutes · 10 slides

SLIDE 1 · Title Slide

Timing: 0:00 – 0:30

Hello everyone. Today I want to talk about FastAPI — specifically how we used it as the central gateway in a microservice system we built for automating the processing of DNA sequencing data.

The system monitors sequencing machines, detects when a run completes, and automatically validates and uploads the output to a secure research environment. But the architecture patterns I'll show you are general purpose — they apply to any system where you need to coordinate a web API, a database, background workers, and an event-driven pipeline.

SLIDE 2 · Why FastAPI?

Timing: 0:30 – 1:30

Let me start with why we chose FastAPI over Flask or Django.

FastAPI is built on top of Starlette, which is an ASGI framework. ASGI means it handles asynchronous I/O natively — you can write `async def` route handlers and the framework will handle concurrent requests without blocking.

But honestly the bigger win for us was the type system integration. FastAPI uses Python type annotations not just as documentation, but as executable contracts. When you declare a route parameter as a `UUID`, FastAPI parses and validates it before your handler runs. When you declare a `response_model`, FastAPI serialises your return value through that schema and strips anything you didn't explicitly expose.

This is powered by Pydantic, which is the validation library FastAPI builds on. Together they give you something that Flask doesn't: a guarantee that if your route handler runs at all, the inputs are already valid.

And as a bonus, all of this type information feeds into automatic OpenAPI documentation — Swagger UI and ReDoc — generated at runtime with zero extra work.

[Note: Pause briefly after 'inputs are already valid' — it's the key point of the slide.]

SLIDE 3 · System Architecture

Timing: 1:30 – 2:30

Here is the full system. Seven Docker Compose services, all running together.

The backbone is the backend FastAPI service — the gateway — running on port 8000. It owns the database, the business logic, and the event publishing. PostgreSQL sits behind it.

The frontend is also a FastAPI service, on port 8001. It renders HTML using Jinja2 templates and calls the backend over HTTP using the `httpx` library. So the frontend is a thin client — it holds no data and has no database connection of its own.

Then we have RabbitMQ as the message broker. The backend publishes events into RabbitMQ, and the worker service consumes them. The worker does the heavy lifting: generating checksums, uploading folders to S3.

Finally, the watcher service monitors the filesystem — the actual directories where DNA sequencing machines write their output files. When it detects that a run has finished, it publishes an event into RabbitMQ, and the whole pipeline kicks off automatically.

[Note: Walk through the diagram on screen left to right, top to bottom.]

SLIDE 4 · FastAPI Routing & Path Parameters

Timing: 2:30 – 3:30

Let me show you what FastAPI route handlers actually look like in this project.

Here we have two routes on the `runs` resource. The first is a standard GET by UUID. Notice that `uuid` is declared as `UUID` type — FastAPI parses the path segment and gives us a Python `UUID` object, or returns a 422 error automatically if it is malformed.

The `db` parameter uses `Depends(get_db)` — that is FastAPI's dependency injection. `get_db` is a generator function that opens a database session, yields it into the handler, and closes it when the handler returns. So the route handler never manages the DB session lifecycle itself.

The second route is more interesting. It is the manual trigger to start uploading a run to the secure research environment. Before publishing the event, it checks that the run is in a valid state — either `running_finished` or `move_failed`. If not, it returns a 409 Conflict. This is the API enforcing the state machine.

The `response_model` on both routes means FastAPI filters the return value through the Pydantic schema — so even if the ORM object has sensitive internal fields, only what is declared in `ShowRun` is returned.

[Note: Point to each highlighted line in the code block as you mention it.]

SLIDE 5 · Pydantic: Validation & Serialisation

Timing: 3:30 – 4:30

Let me dig into the schemas a bit more.

We have separate schemas for creating a sequencer versus showing one. `SequencerCreate` is what the client sends — it includes the name, location, and a `sent_to_tre` field. That field is typed as `Literal["auto", "manual"]` — so FastAPI will reject any request that sends something

outside those two values, before it ever reaches the database.

ShowSequencer is what we return. It uses `model_config = ConfigDict(from_attributes=True)` — that tells Pydantic it can read attribute values directly from a SQLAlchemy ORM object. So we can return the ORM object from the route handler and Pydantic pulls the fields off automatically.

The ShowRun schema also includes `sequencer_name` — a field that doesn't exist directly on the runs table. It comes from the ORM relationship join. Because of `from_attributes=True`, Pydantic traverses the relationship and pulls the name from the linked sequencer row. This is very clean — the route handler just returns the ORM object, and the schema handles all the serialisation logic.

SLIDE 6 · Database Layer: SQLAlchemy + Repository Pattern

Timing: 4:30 – 5:30

For the database, we use SQLAlchemy 2.0 with a repository pattern.

Every table has two kinds of identifiers. An integer id that is the primary key internally — fast for joins. And a uuid column that is what we expose in the API. This means external clients never see database-internal IDs. If we ever need to shard or re-sequence, the external interface is stable.

All deletes are soft deletes — we set `is_deleted` to True and filter it out in queries. This is important for audit trails in a regulated research context.

The repository pattern means all database operations are in dedicated functions, not scattered across route handlers. The route handler calls `run_repo.update_run_status(db, uuid, 'checksumming')` — it doesn't write SQL. This makes the routes easy to read and the DB logic easy to test in isolation.

Schema migrations are managed with Alembic, which integrates cleanly with SQLAlchemy. Each migration is a versioned Python file describing the forward and rollback changes.

SLIDE 7 · Async Event-Driven Communication

Timing: 5:30 – 6:30

Now let me talk about what happens after the route handler returns.

When a user clicks 'Send to TRE', the backend publishes a `run_checksum_requested` event to RabbitMQ and immediately returns the updated run to the frontend. The HTTP response is instant. The actual work — generating checksums and uploading gigabytes of data — happens in the worker container, asynchronously.

The event is a simple JSON message with three fields: the event type, the run name, and the sequencer UUID. The worker subscribes to the queue with `pika`, the Python RabbitMQ client, and dispatches based on the event type.

The full automated pipeline looks like this: the watcher detects the completion signal file and publishes `run_completed`. The worker receives it and sets the status to `running_finished`. If the sequencer is configured for automatic processing, it immediately publishes `run_checksum_requested`. The worker checksums the folder, then publishes `run_upload_requested`. The worker uploads to S3 and sets the status to `completed`.

Every single one of those status transitions is recorded in the `runs_status_history` table — so we have a permanent, timestamped audit trail for every run.

[Note: Read the pipeline flow on screen slowly — give the audience time to follow it.]

SLIDE 8 · Filesystem Watcher as an Event Source

Timing: 6:30 – 7:30

One of the more unusual parts of this system is the watcher service. It bridges the physical world — DNA sequencing machines writing files to disk — to our event-driven pipeline.

We use the `watchdog` Python library. It wraps platform filesystem notification APIs. But we are running inside Docker on macOS, and Docker Desktop doesn't forward filesystem events from the host to containers. So we use `PollingObserver` — it polls the directories every 10 seconds. Less efficient than native events, but it works reliably across all platforms.

The completion signal is configurable per sequencer type. In the database, each sequencer type stores a `completion_signal` string and a `signal_match` mode: `exact`, `prefix`, or `suffix`. So for Illumina machines, the signal is `RTAComplete.txt` — an exact match. The watcher checks each new file against the configured signal for that sequencer.

When a match is found, the watcher calls the same `publisher.publish()` function the API uses. The run name is just the parent folder name. That's enough — the worker looks up the rest from the database.

[Note: Mention: watcher also polls DB every 60 s — no restart needed when adding a new sequencer.]

SLIDE 9 · Two FastAPI Services: API + Frontend

Timing: 7:30 – 8:30

Let me say a word about the split between the two FastAPI services, because it is a deliberate architectural choice.

The backend is a pure REST API. It speaks JSON only. It has no concept of HTML, no session management, no cookies. Anyone can call it — the web frontend, a CLI tool, another service, a notebook.

The frontend is also FastAPI, but it plays a different role. It is a server-side rendering layer. Its route handlers call the backend over HTTP using `httpx`, get back JSON, and pass it to Jinja2 templates which render HTML. The frontend has no database connection — it is entirely dependent on the backend API.

This separation has real benefits. The backend can evolve independently. Other consumers can be added without touching the frontend. And if you want to replace the Jinja2 frontend with a React app, you discard one service and the backend doesn't change at all.

Both services share the same patterns: FastAPI route handlers, Pydantic schemas, structured logging with rotating file handlers, and a global exception handler that logs unhandled errors before returning a 500 response.

SLIDE 10 · Key Takeaways

Timing: 8:30 – 10:00

Let me close with the key takeaways.

FastAPI earns its keep in this project not just as a request router, but as the central coordinator of the entire system. It validates inputs before they touch the database. It enforces business rules — the state machine — at the HTTP layer. It publishes events that drive the async pipeline. And it exposes clean, documented JSON contracts that the frontend and any future client can consume.

The patterns we used are all standard FastAPI idioms: Pydantic schemas as contracts, dependency injection for sessions and config, `response_model` to control what gets returned, and `HTTPException` for structured error responses.

The async event-driven side — RabbitMQ, the worker, the watcher — is where the system scales. The API stays fast because it hands off heavy work immediately. The worker processes at its own pace. If upload volume spikes, we can run multiple worker containers without changing the API.

And because every event and every status transition is recorded, we have a complete audit trail — which matters a lot in a regulated research environment.

That is the system. FastAPI as a gateway: thin, fast, type-safe, and ready to hand off to async infrastructure when the real work begins. Thank you. Happy to take questions.

[Note: Allow extra time here for questions. Suggested: 1–2 minutes buffer.]

Total estimated delivery time: ~10 minutes at a moderate pace. Code slides (4, 5, 6, 7, 8): allow a few extra seconds for the audience to read snippets.